

# Exercícios de Oracle PL/SQL

Blocos anônimos: variáveis, estruturas condicionais ( `IF` ) e estruturas de repetição ( `LOOP` )

Lista de exercícios • Nível introdutório • Requisito: habilitar `SET SERVEROUTPUT ON` antes de executar os blocos

Aluno(a): \_\_\_\_\_ Data: \_\_\_\_/\_\_\_\_/\_\_\_\_

## Exercício 1 — Tabuada com validação de entrada

**Objetivo:** praticar declaração de variáveis, estrutura condicional `IF ... ELSE` e repetição com `FOR LOOP`.

### CONTEXTO

Você foi contratado para criar um programa que gera a tabuada de um número escolhido, mas o sistema só aceita números de 1 a 10.

### O QUE O BLOCO ANÔNIMO DEVE FAZER

- Na seção `DECLARE`, crie:
  - uma variável para armazenar o número da tabuada (tipo numérico), já inicializada com um valor de sua escolha;
  - uma variável para armazenar o resultado de cada multiplicação.
- Na seção `BEGIN`, verifique se o número informado está **fora** do intervalo de 1 a 10.
  - Se estiver fora, exiba a mensagem `Erro: informe um número entre 1 e 10.` e **não** calcule nada.
  - Se estiver dentro, exiba um cabeçalho no formato `Tabuada do X` e depois gere as 10 linhas da tabuada.
- Use um `FOR LOOP` de 1 a 10 para calcular e imprimir cada linha.
- Toda a saída deve usar `DBMS_OUTPUT.PUT_LINE`.

### FORMATO ESPERADO DA SAÍDA (EXEMPLO COM O NÚMERO 7)

```
Tabuada do 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

### DICAS

- Lembre-se de habilitar `SET SERVEROUTPUT ON` antes de executar.
- Para concatenar texto com números use o operador `||`.
- Teste seu bloco duas vezes: uma com um valor válido (ex.: 7) e outra com um valor inválido (ex.: 15), para confirmar que a validação funciona.

### DESAFIO EXTRA (OPCIONAL)

Troque o valor fixo por uma variável de substituição ( `&numero` ) e refaça os testes.

## Exercício 2 — Boletim da turma com classificação de notas

**Objetivo:** praticar múltiplas variáveis, acumuladores, contadores, `IF ... ELSIF ... ELSE` e repetição com `WHILE LOOP`.

### CONTEXTO

Uma escola precisa processar as notas finais de uma turma de 5 alunos, classificar cada aluno e apresentar um resumo da turma.

### O QUE O BLOCO ANÔNIMO DEVE FAZER

- Na seção `DECLARE`, crie:
  - um contador que controlará qual aluno está sendo processado (comece em 1);
  - uma variável com o total de alunos (5);
  - uma variável para a nota do aluno atual (numérica, com 2 casas decimais);
  - uma variável acumuladora para somar todas as notas (inicie em 0);
  - um contador de aprovados (inicie em 0);
  - uma variável para a média final da turma.
- Use um `WHILE LOOP` que repita enquanto o contador for menor ou igual ao total de alunos.
- Dentro do laço, para cada aluno:
  - gere uma nota entre 0 e 10 com `ROUND(DBMS_RANDOM.VALUE(0, 10), 2)`;
  - some essa nota ao acumulador;
  - classifique a nota com `IF / ELSIF / ELSE` segundo a regra:
    - nota **maior ou igual a 7** → `APROVADO` (e incremente o contador de aprovados);
    - nota **maior ou igual a 5 e menor que 7** → `RECUPERAÇÃO`;
    - nota **menor que 5** → `REPROVADO`;
  - imprima uma linha no formato `Aluno N - Nota: X - SITUAÇÃO`;
  - não esqueça de incrementar o contador** — se esquecer, o laço nunca termina.
- Após o laço, calcule a média (soma dividida pelo total de alunos, arredondada com 2 casas) e exiba:
  - uma linha separadora;
  - a média da turma;
  - o total de alunos aprovados.

### FORMATO ESPERADO DA SAÍDA

```
Aluno 1 - Nota: 8.45 - APROVADO
Aluno 2 - Nota: 5.2 - RECUPERAÇÃO
Aluno 3 - Nota: 3.9 - REPROVADO
Aluno 4 - Nota: 9.1 - APROVADO
Aluno 5 - Nota: 6.7 - RECUPERAÇÃO
-----
Média da turma: 6.67
Total de aprovados: 2
```

### DICAS

- A ordem das condições no `IF/ELSIF` importa: teste primeiro a faixa mais alta, senão a lógica fica errada.
- Como as notas são aleatórias, cada execução produz um resultado diferente — execute algumas vezes e confira se as classificações estão coerentes.
- Use `PLS_INTEGER` para contadores (é mais eficiente que `NUMBER` para números inteiros).

### **DESAFIO EXTRA (OPCIONAL)**

Guarde também a maior e a menor nota da turma e exiba-as no resumo final.

## Exercício 3 — Identificação de números primos

**Objetivo:** praticar `LOOP` básico com `EXIT WHEN`, laços aninhados, variável do tipo `BOOLEAN` e construção de string acumulada.

### CONTEXTO

Você precisa montar uma rotina que descubra todos os números primos existentes até um determinado limite. Um número é **primo** quando é maior que 1 e só é divisível por 1 e por ele mesmo.

### O QUE O BLOCO ANÔNIMO DEVE FAZER

- Na seção `DECLARE`, crie:
  - uma variável com o limite superior da busca (ex.: 30);
  - uma variável para o número que está sendo testado (comece em 2, o menor primo);
  - uma variável para o divisor usado nos testes;
  - uma variável `BOOLEAN` que indica se o número atual é primo;
  - um contador da quantidade de primos encontrados;
  - uma variável `VARCHAR2` para ir montando a lista dos primos.
- Use um `LOOP` básico (sem `FOR` e sem `WHILE`) com `EXIT WHEN` para percorrer os números de 2 até o limite.
- Para cada número testado:
  - assuma inicialmente que ele **é** primo (variável booleana em `TRUE`) e reinicie o divisor em 2;
  - use um **laço interno** para testar os divisores. Basta testar divisores até a raiz quadrada do número (`SQRT`), pois além disso não existem novos divisores possíveis;
  - se `MOD(numero, divisor) = 0`, o número **não** é primo: marque a variável booleana como `FALSE` e interrompa o laço interno com `EXIT` (não faz sentido continuar testando);
  - após o laço interno, se a variável booleana continuar `TRUE`, incremente o contador e concatene o número na variável de lista, separado por espaço;
  - incremente o número testado antes de voltar ao topo do laço externo.
- Ao final, exiba a lista completa dos primos encontrados e a quantidade total.

### FORMATO ESPERADO DA SAÍDA (LIMITE = 30)

```
Primos até 30: 2 3 5 7 11 13 17 19 23 29
Quantidade: 10
```

### DICAS

- Em PL/SQL, um `BOOLEAN` é testado diretamente: `IF v_primo THEN ...` (não use `= TRUE`).
- `BOOLEAN` **não pode** ser concatenado no `DBMS_OUTPUT` — use-o apenas dentro do `IF`.
- O `EXIT` sem `WHEN` sai do laço mais interno em que está; o `EXIT WHEN condição` só sai quando a condição for verdadeira.
- Cuidado com a atualização das variáveis de controle nos dois laços — esquecer uma delas gera loop infinito. Se isso acontecer, interrompa a execução na sua IDE.

### DESAFIO EXTRA (OPCIONAL)

Altere o limite para 100 e verifique se o programa encontra 25 primos. Depois, use rótulos de laço ( <<externo>> ) e experimente `EXIT externo WHEN ...`.

## RESUMO DOS CONTEÚDOS PRATICADOS

| Ex. | Variáveis                           | Condicional           | Repetição                   |
|-----|-------------------------------------|-----------------------|-----------------------------|
| 1   | NUMBER                              | IF ... ELSE           | FOR LOOP                    |
| 2   | PLS_INTEGER , NUMBER(4,2)           | IF ... ELSIF ... ELSE | WHILE LOOP                  |
| 3   | PLS_INTEGER , BOOLEAN ,<br>VARCHAR2 | IF + EXIT WHEN        | LOOP básico + laço aninhado |